Serverless File Processing and Tracking System

Executive Summary

This project implements a small serverless application for accepting text-file uploads and tracking their processing status conveniently. It is intended for a user or small team that requires a straightforward way to submit a file and retrieve a status record without managing servers. Amazon API Gateway provides HTTP endpoints, AWS Lambda handles requests, Amazon S3 stores uploaded file content, Amazon DynamoDB stores tracking metadata, Amazon CloudWatch captures execution logs and finally AWS CloudFormation defines the infrastructure as code. The project focuses on a simple request-driven design with services that are managed and a reproducible deployment. The repository includes the CloudFormation template, README instructions, an architecture diagram, and evidence files. The documented test uploaded demo.txt, then returned a PROCESSED status and verified the uploaded object in S3 and the tracking record in DynamoDB. The design caters for a low operational burden and pay-per-use components over a continuously running server. The main trade-offs are that the current implementation is a compact demonstration rather than a production-grade file-processing platform. Authentication, stronger input controls and more extensive failure handling would be appropriate next steps before exposing it to real users.

Architecture and Service Choices

The request path begins with a client sending an HTTP request to API Gateway. A POST request to /files gives a filename and text content. API Gateway invokes the Lambda function which validates the request, creates a unique file identifier, writes the file to S3, and stores a tracking item in DynamoDB. The response includes the identifier and initial processing status. A GET request to /status/{fileId} retrieves the stored record for that identifier.

Amazon API Gateway — HTTP interface

- API Gateway exposes the upload and status routes and connects HTTP requests to Lambda using proxy integration. This keeps the application interface separate from the implementation code and avoids operating a web server. The current template configures the methods with no authorization, which is acceptable only for a controlled demonstration and should be improved before production use.

AWS Lambda — Request processing

- Lambda runs the Python 3.12 handler on demand. It reads the filename and content from the request, generates a UUID, writes the file and records the filename, S3 key, status and creation timestamp. The configured function utilizes a 15-second timeout and 128 MB of memory. Lambda avoids provisioning or patching an always-on compute instance. Its restrictions and error behavior still require to be considered for larger files or heavier workloads.

Amazon S3 — Uploaded file storage

- S3 stores the uploaded text object under a key built from the unique file ID and filename. The template enables S3-managed server-side encryption (AES-256) and all four public-access-block settings. These settings reduce the risk of

accidental public exposure. The current function role allows object writes to the bucket; production access should be reviewed for least privilege and any additional read, retention, or lifecycle needs.

Amazon DynamoDB — Tracking records

- DynamoDB stores one item per file, keyed by the string attribute field. The item includes the filename, S3 object key, status, and UTC creation time. On-demand billing avoids capacity planning for a small or unpredictable workload. The design is simple to query by file ID, although reporting across dates or statuses would require additional indexes or a separate analytics approach.

Amazon CloudWatch and AWS CloudFormation

- CloudWatch Logs capture Lambda execution output for troubleshooting. CloudFormation defines the bucket, table, IAM role, function, API resources, deployment, permissions and outputs in one template. Infrastructure as code makes resource configuration reviewable and repeatable and lets the stack be removed after evidence capture. The IAM role separates Lambda permissions from a user's own AWS console permissions.

Implementation, Configuration, and Testing

The deployment is documented for AWS Region us-west-2. The repository's README gives the CloudFormation deployment command and identifies the template file, stack name and IAM capability flag. The main infrastructure definition is template.yaml. The repository also includes the README, architecture documentation, and evidence folder.

Key configuration choices

- S3 bucket public-access blocking is enabled, and default server-side encryption utilizes AES-256.
- DynamoDB utilizes on-demand billing and a string partition key named field.
- Lambda uses Python 3.12, 128 MB memory, a 15-second timeout and environment variables for the bucket and table names.
- The API is deployed to a prod stage with POST upload and GET status methods.
- The Lambda execution role grants log-writing permissions, S3 object-write permission and DynamoDB item read/write permissions for the configured table.

Testing and evidence

The test documented in the README submits demo.txt with text content to the upload endpoint. The response reports PROCESSED; the object is checked in S3, and the tracking item is checked in DynamoDB. These checks confirm the main end-to-end path across the API, Lambda, object storage and metadata store. The evidence folder should contain legible screenshots that collectively show the successful request or CLI output, relevant AWS resources, and the application output with the name visible.

Implementation considerations and challenges

A central implementation consideration is keeping the API routes, Lambda event parsing and CloudFormation resources consistent. The handler distinguishes POST requests to /files from GET requests to /status/{fileId}, so route configuration and event fields must match the code's expectations. The unique ID associates the S3 object with the DynamoDB record. Another consideration is making evidence reproducible: the README records the deployment command, region, test outcome, and cleanup instruction so another user can understand the workflow.

Architectural Concerns

Security

The current template contains useful baseline controls. S3 public access is blocked, uploaded objects are encrypted at rest, and Lambda utilizes a dedicated execution role. However, the API methods utilize Authorization Type: NONE, meaning anyone who can reach the endpoint may be able to submit requests or query a known file ID. Before production use, add authentication and authorization, validate request size and content more strictly, review IAM permissions for least privilege and consider API throttling and monitoring. Avoid placing secrets or sensitive file content in logs.

Reliability

Managed services reduce the requirement to maintain servers and can scale within service limits. DynamoDB records provide a durable status lookup and CloudWatch Logs support investigation when requests fail. The current implementation writes to S3 and then DynamoDB; if the first operation succeeds but the second fails, the system may leave an object without a matching tracking record. A stronger design would add explicit error handling, retries wherever safe and a way to reconcile incomplete

operations. Automated tests should cover malformed requests, missing IDs, missing records, and partial failures.

Cost optimization and performance

Lambda's on-demand execution, API Gateway and DynamoDB on-demand billing suit a low-volume demonstration because there is no server to keep running. S3 storage and service requests can still incur charges, so uploads should have reasonable limits, and a lifecycle policy may assist if files accumulate. Performance should be tested with representative file sizes and request volumes rather than assumed from one successful upload. If usage grows, monitor Lambda duration/errors, API latency and throttles, DynamoDB capacity, and S3 storage/request costs.

Well-Architected Review

Three-pillar self-review

Operational excellence: Infrastructure is defined in CloudFormation, and setup steps are documented in GitHub, improving repeatability. The next improvement is automated deployment validation and a repeatable test script that can run after each modification.

Security: Public S3 access is blocked, and data is encrypted by default. The biggest identified gap is the unauthenticated API. Authorization, tighter validation, and a least-privilege review should be prioritized before real-world use.

Reliability: The core upload-and-record flow was tested across S3 and DynamoDB and logs are available for Troubleshooting. More comprehensive failure-path tests, retry behavior and reconciliation for partial writes would improve resilience.

What I Would Do Differently

If I rebuilt the project, I would design the failure paths and security model earlier instead of focusing first on the successful upload. I would require authentication for the API, define more limitative permissions and add input limits and clearer validation errors. I would also add automated tests for valid uploads, invalid payloads, and unknown file IDs between S3 and DynamoDB. A more robust processing workflow could use an asynchronous event-driven step and explicit status transitions rather than immediately marking every accepted file as PROCESSED. Finally, I would add cost and operational alarms, document expected limitations and make the evidence capture the checklist part of the project workflow.

Cleanup

After capturing the required screenshots and verifying the project evidence, I deleted the file-tracking-project CloudFormation stack in the us-west-2 AWS region. This removed the AWS resources managed by the stack and assisted to prevent unnecessary ongoing charges.